Nonparametric prediction

Lecture 9 Neighbors, trees, and forests

Tao LIN

SoE, Westlake University

November 3, 2026



Fitting is only part of learning

How do we predict well on unseen data?

Model
Choose a family

→

Objective
Define a good fit

→

Fit
Learn from data

→

Generalize
Predict on new data

More flexible does not always mean more reliable

L02



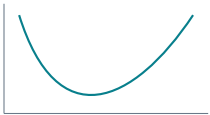
Fit a model

L03



Add flexibility

L04–05



Choose by validation

More flexible does not always mean more reliable

L02



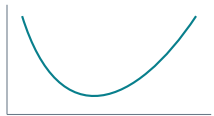
Fit a model

L03



Add flexibility

L04–05



Choose by validation

Better training fit $\not\Rightarrow$ better prediction.

Probability gives familiar losses a common language

Gaussian response

$Y \mid x \sim \mathcal{N}(w^\top x, \sigma^2)$

$\xrightarrow{\text{MLE}}$ **Squared error**

Bernoulli response

$P(Y = 1 \mid x) = \sigma(w^\top x)$

$\xrightarrow{\text{MLE}}$ **Cross-entropy**

Probability gives familiar losses a common language

Gaussian response

$Y \mid x \sim \mathcal{N}(w^\top x, \sigma^2)$

$\xrightarrow{\text{MLE}}$ **Squared error**

Bernoulli response

$P(Y = 1 \mid x) = \sigma(w^\top x)$

$\xrightarrow{\text{MLE}}$ **Cross-entropy**

L06: GLMs connect the response distribution to the input.

Labels change what we can learn

Labels observed

GDA

$$p(x, y) = p(y)p(x \mid y)$$

Predict a class

hide y →

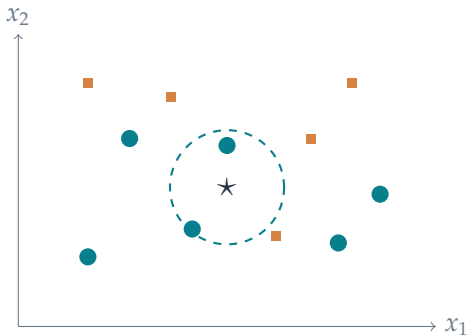
Membership hidden

GMM + EM

$$p(x) = \sum_k \pi_k p(x \mid z = k)$$

Discover components

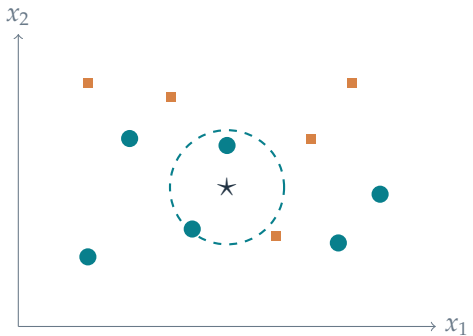
Can nearby labels supply the prediction?



KNN

Choose nearby examples.

Can nearby labels supply the prediction?



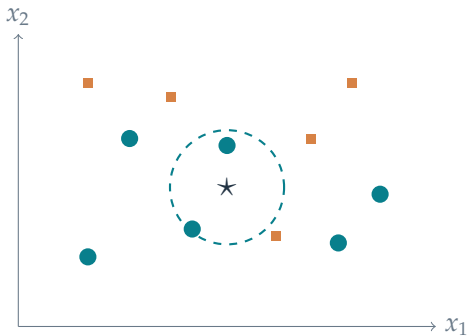
KNN

Choose nearby examples.

Tree

Learn prediction regions.

Can nearby labels supply the prediction?



KNN

Choose nearby examples.

Tree

Learn prediction regions.

Forest

Combine different trees.

The predictor can grow with the data

PARAMETRIC

A fixed-size parameter vector

Fixed features, fitted weights.

NONPARAMETRIC

Data-dependent complexity

Neighbors or growing partitions.

The predictor can grow with the data

PARAMETRIC

A fixed-size parameter vector

Fixed features, fitted weights.

NONPARAMETRIC

Data-dependent complexity

Neighbors or growing partitions.

Both still need assumptions and validation.

K-Nearest Neighbors (KNN)

- **Core Idea:** “Similar inputs should have similar outputs.”

K-Nearest Neighbors (KNN)

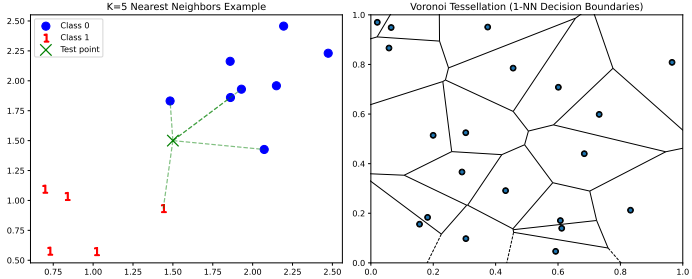
- **Core Idea:** “Similar inputs should have similar outputs.”
- To classify a new point $\mathbf{x}$, look at the K points in the training data that are **closest** to it.

K-Nearest Neighbors (KNN)

- **Core Idea:** “Similar inputs should have similar outputs.”
- To classify a new point $\mathbf{x}$, look at the K points in the training data that are **closest** to it.
- This is a memory-based or lazy learning algorithm.
  - **No “training” phase:** It just memorizes the entire training set $\mathcal{D}$.
  - **All computation happens at prediction time**.

K-Nearest Neighbors (KNN)

- **Core Idea:** “Similar inputs should have similar outputs.”
- To classify a new point $\mathbf{x}$, look at the K points in the training data that are **closest** to it.
- This is a memory-based or lazy learning algorithm.
  - **No “training” phase:** It just memorizes the entire training set $\mathcal{D}$.
  - **All computation happens at prediction time**.



KNN: find neighbors, then aggregate their labels

Distance $\longrightarrow$ K nearest examples $\longrightarrow$ Prediction

Classification: vote

$$\hat{y}(x) = \arg\max_{c} \sum_{i \in \mathcal{N}_K(x)} \mathbf{1}[y_i = c].$$

Regression: average

$$\hat{y}(x) = \frac{1}{K} \sum_{i \in \mathcal{N}_K(x)} y_i.$$

Choose K and a meaningful distance.

KNN: Visual Example

- **K=5 Example**:
  - Test point (X) with 5 nearest neighbors
  - Class probability: $p(y = 1|x, \mathcal{D}) = \frac{3}{5}$
  - Prediction: Class 1 (majority vote)

KNN: Visual Example

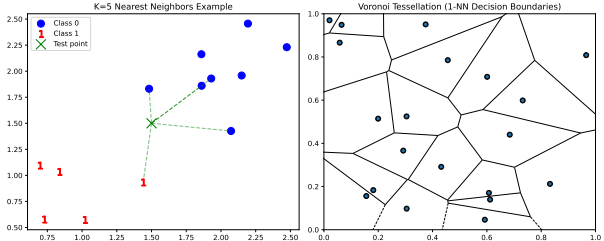
- **K=5 Example**:
  - Test point (X) with 5 nearest neighbors
  - Class probability: $p(y = 1|x, \mathcal{D}) = \frac{3}{5}$
  - Prediction: Class 1 (majority vote)

- **1-NN Decision Boundaries**:
  - Voronoi tessellation
  - Each region belongs to closest training point
  - Boundaries where nearest neighbor changes

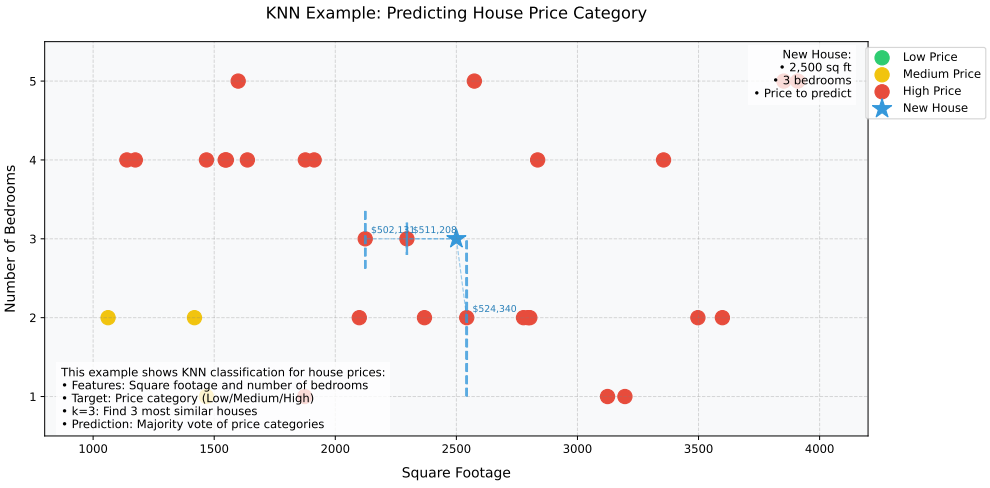
KNN: Visual Example

- **K=5 Example**:
  - Test point (X) with 5 nearest neighbors
  - Class probability: $p(y = 1|x, \mathcal{D}) = \frac{3}{5}$
  - Prediction: Class 1 (majority vote)

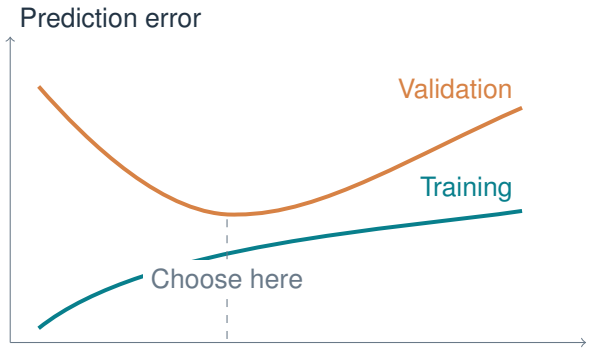
- **1-NN Decision Boundaries**:
  - Voronoi tessellation
  - Each region belongs to closest training point
  - Boundaries where nearest neighbor changes



KNN: Prediction house price categories



Choosing K: flexibility has a cost



Schematic curves; no numerical dataset implied.

Small K: flexible

Less averaging; more sensitive to samples.

Large K: smooth

More averaging; can miss local structure.

Choose K using validation error.

Why does a neighborhood contain a fraction r^d?

Setup: inputs are uniform on $[0, 1]^d$; the neighborhood is a contained cube.

r is its side length, measured relative to each unit-length axis.

One dimension



Length $= r$

Why does a neighborhood contain a fraction r^d?

Setup: inputs are uniform on $[0, 1]^d$; the neighborhood is a contained cube.

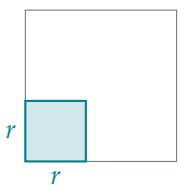
r is its side length, measured relative to each unit-length axis.

One dimension



Length $= r$

Two dimensions



Area $= r \times r$

Why does a neighborhood contain a fraction r^d?

Setup: inputs are uniform on $[0, 1]^d$; the neighborhood is a contained cube.

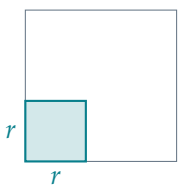
r is its side length, measured relative to each unit-length axis.

One dimension



Length $= r$

Two dimensions



Area $= r \times r$

d dimensions

$$\underbrace{r \times \cdots \times r}_{d \text{ axes}} = r^d$$

Volume $= r^d$

Why does a neighborhood contain a fraction r^d?

Setup: inputs are uniform on $[0,1]^d$; the neighborhood is a contained cube.

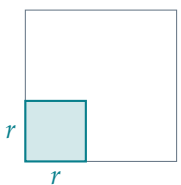
r is its side length, measured relative to each unit-length axis.

One dimension



Length $= r$

Two dimensions



Area $= r \times r$

d dimensions

$$\underbrace{r \times \cdots \times r}_{d \text{ axes}} = r^d$$

Volume $= r^d$

Uniform inputs: probability $=$ volume fraction $= r^d$.

To capture 1%, how wide must the neighborhood be?

Target probability α $=$ cube volume r^d

To capture 1%, how wide must the neighborhood be?

Target probability α $=$ cube volume r^d

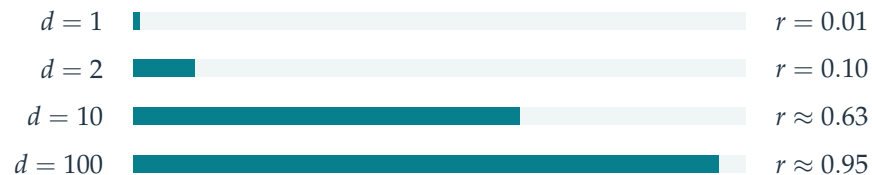
$$r^d = \alpha \quad \Longrightarrow \quad r = \alpha^{1/d}.$$

To capture 1%, how wide must the neighborhood be?

Target probability α $=$ cube volume r^d

$$r^d = \alpha \quad \Longrightarrow \quad r = \alpha^{1/d}.$$

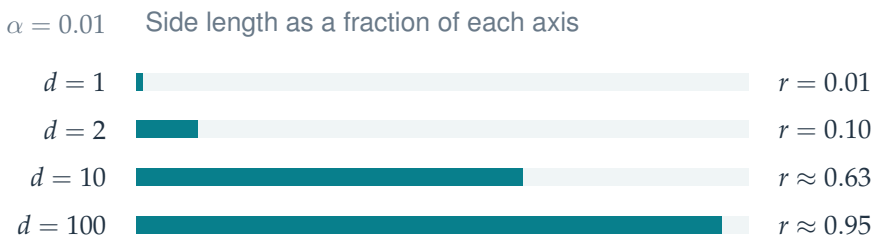
$\alpha = 0.01$ Side length as a fraction of each axis



To capture 1%, how wide must the neighborhood be?

Target probability α = cube volume r^d

$$r^d = \alpha \quad \Longrightarrow \quad r = \alpha^{1/d}.$$



The same 1% becomes much less local as d grows.

Local coverage can require exponentially more data

Same uniform example: for a contained cube of side r, independence gives

$$P(\text{at least one training point inside}) = 1 - (1 - r^d)^N.$$

Local coverage can require exponentially more data

Same uniform example: for a contained cube of side r, independence gives

$$P(\text{at least one training point inside}) = 1 - (1 - r^d)^N.$$

Setting this probability to $1/2$ yields

$$r = \left(1 - 2^{-1/N}\right)^{1/d}.$$

Local coverage can require exponentially more data

Same uniform example: for a contained cube of side r, independence gives

$$P(\text{at least one training point inside}) = 1 - (1 - r^d)^N.$$

Setting this probability to $1/2$ yields

$$r = \left(1 - 2^{-1/N}\right)^{1/d}.$$

For $N = 500$ and $d = 10$, $r \approx 0.52$: even finding one point can require a broad region.

Local coverage can require exponentially more data

Same uniform example: for a contained cube of side r, independence gives

$$P(\text{at least one training point inside}) = 1 - (1 - r^d)^N.$$

Setting this probability to $1/2$ yields

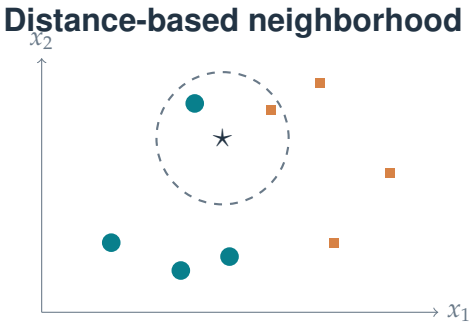
$$r = \left(1 - 2^{-1/N}\right)^{1/d}.$$

For $N = 500$ and $d = 10$, $r \approx 0.52$: even finding one point can require a broad region.

For a fixed side length $0 < r < 1$, achieving 50% coverage requires $N \geq \log(1/2)/\log(1 - r^d) \approx (\log 2) r^{-d}$ when r^d is small.

Can labels tell us which region to use?

KNN: choose neighbors using a distance.



Nearby examples can mix labels.

Can labels tell us which region to use?

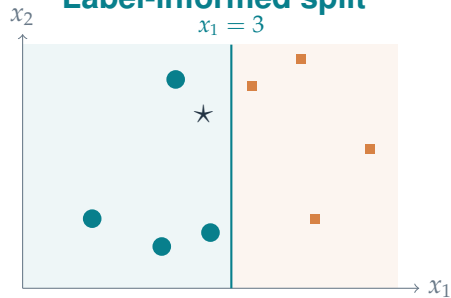
KNN: choose neighbors using a distance. **Tree:** learn regions using labels.

Distance-based neighborhood



Nearby examples can mix labels.

Label-informed split



One threshold separates these labels.

Can labels tell us which region to use?

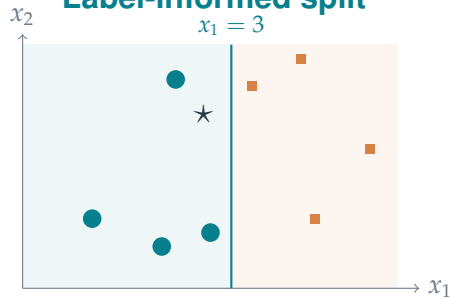
KNN: choose neighbors using a distance. **Tree:** learn regions using labels.

Distance-based neighborhood



Nearby examples can mix labels.

Label-informed split



One threshold separates these labels.

Which feature should we split on, and where?

Decision Trees

- A different approach that is more structured than KNN.

Decision Trees

- A different approach that is more structured than KNN.
- **Core Idea:** Recursively partition the feature space by asking a series of simple (axis-aligned) questions.

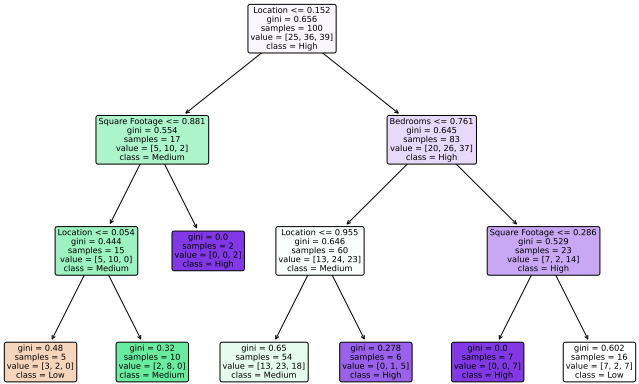
Decision Trees

- A different approach that is more structured than KNN.
- **Core Idea:** Recursively partition the feature space by asking a series of simple (axis-aligned) questions.
- This creates a tree structure:
  - **Internal Nodes:** A question about a feature (e.g., “Is ‘sq. footage‘ > 1500?”).
  - **Branches:** The answer to the question (e.g., “Yes” or “No”).
  - **Leaf Nodes:** A final prediction (e.g., “Expensive” or “Avg. Price”).

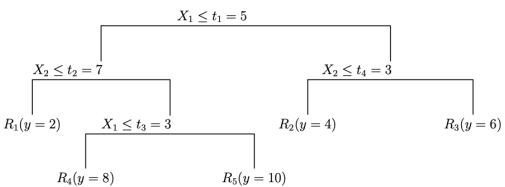
Decision Trees

- A different approach that is more structured than KNN.
- **Core Idea:** Recursively partition the feature space by asking a series of simple (axis-aligned) questions.
- This creates a tree structure:
  - **Internal Nodes:** A question about a feature (e.g., “Is ‘sq. footage‘ > 1500?”).
  - **Branches:** The answer to the question (e.g., “Yes” or “No”).
  - **Leaf Nodes:** A final prediction (e.g., “Expensive” or “Avg. Price”).
- This is a white-box model: the decision logic is highly interpretable.

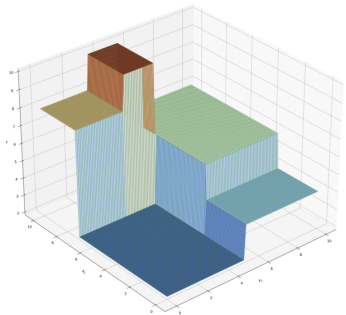
Decision Trees



Decision Trees: Regression Tree



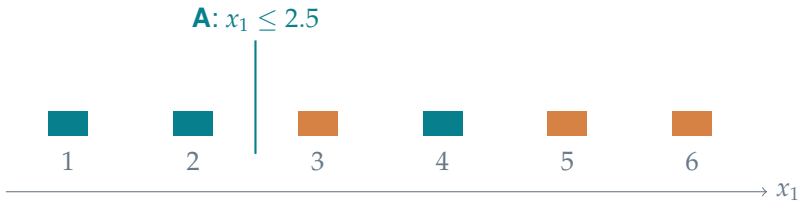
(a)



(b)

Where should we split these six samples?

Same training node, two candidate thresholds on x_1.



Where should we split these six samples?

Same training node, two candidate thresholds on x_1.



Where should we split these six samples?

Same training node, two candidate thresholds on x_1.



A makes one child pure. B leaves both children mixed.

How do we compare the two splits fairly?

Score the split: how mixed are the children?

Gini impurity: $G = 1 - p_{\text{blue}}^2 - p_{\text{orange}}^2$

A: split after sample 2

Left: 2 blue, 0 orange $\quad G_L = 0$

Right: 1 blue, 3 orange

$$G_R = 1 - (1/4)^2 - (3/4)^2 = 3/8.$$

B: split after sample 3

Left: 2 blue, 1 orange

Right: 1 blue, 2 orange

$$G_L = G_R = 1 - (2/3)^2 - (1/3)^2 = 4/9.$$

Score the split: how mixed are the children?

Gini impurity: $G = 1 - p_{\text{blue}}^2 - p_{\text{orange}}^2$

A: split after sample 2

Left: 2 blue, 0 orange $\quad G_L = 0$

Right: 1 blue, 3 orange

$$G_R = 1 - (1/4)^2 - (3/4)^2 = 3/8.$$

Weighted score:

$$\frac{2}{6}(0) + \frac{4}{6}\left(\frac{3}{8}\right) = \boxed{\frac{1}{4}}.$$

B: split after sample 3

Left: 2 blue, 1 orange

Right: 1 blue, 2 orange

$$G_L = G_R = 1 - (2/3)^2 - (1/3)^2 = 4/9.$$

Weighted score:

$$\frac{3}{6}\left(\frac{4}{9}\right) + \frac{3}{6}\left(\frac{4}{9}\right) = \boxed{\frac{4}{9}}.$$

Score the split: how mixed are the children?

Gini impurity: $G = 1 - p_{\text{blue}}^2 - p_{\text{orange}}^2$

A: split after sample 2

Left: 2 blue, 0 orange $\quad G_L = 0$

Right: 1 blue, 3 orange

$$G_R = 1 - (1/4)^2 - (3/4)^2 = 3/8.$$

Weighted score:

$$\frac{2}{6}(0) + \frac{4}{6}\left(\frac{3}{8}\right) = \boxed{\frac{1}{4}}.$$

B: split after sample 3

Left: 2 blue, 1 orange

Right: 1 blue, 2 orange

$$G_L = G_R = 1 - (2/3)^2 - (1/3)^2 = 4/9.$$

Weighted score:

$$\frac{3}{6}\left(\frac{4}{9}\right) + \frac{3}{6}\left(\frac{4}{9}\right) = \boxed{\frac{4}{9}}.$$

Prefer A: lower weighted impurity ($0.25 < 0.44$).

Decision Trees: Pros and Cons

Advantages

Disadvantages

Decision Trees: Pros and Cons

Advantages

- **Highly interpretable**.

Disadvantages

- **High Variance:** Very prone to overfitting. A small change in data can lead to a completely different tree.

Decision Trees: Pros and Cons

Advantages

- **Highly interpretable**.
- Can handle numerical and categorical data.

Disadvantages

- **High Variance:** Very prone to overfitting. A small change in data can lead to a completely different tree.
- Greedy algorithm finds a local optimum, not guaranteed to be the *best* tree.

Decision Trees: Pros and Cons

Advantages

- **Highly interpretable**.
- Can handle numerical and categorical data.
- No feature scaling (e.g., normalization) is required.

Disadvantages

- **High Variance:** Very prone to overfitting. A small change in data can lead to a completely different tree.
- Greedy algorithm finds a local optimum, not guaranteed to be the *best* tree.
- Can struggle with non-axis-aligned boundaries.

Decision Trees: Pros and Cons

Advantages

- **Highly interpretable**.
- Can handle numerical and categorical data.
- No feature scaling (e.g., normalization) is required.
- Fast for prediction (just follow a path).

Disadvantages

- **High Variance:** Very prone to overfitting. A small change in data can lead to a completely different tree.
- Greedy algorithm finds a local optimum, not guaranteed to be the *best* tree.
- Can struggle with non-axis-aligned boundaries.

Decision Trees: Pros and Cons

Advantages

- **Highly interpretable**.
- Can handle numerical and categorical data.
- No feature scaling (e.g., normalization) is required.
- Fast for prediction (just follow a path).

Disadvantages

- **High Variance:** Very prone to overfitting. A small change in data can lead to a completely different tree.
- Greedy algorithm finds a local optimum, not guaranteed to be the *best* tree.
- Can struggle with non-axis-aligned boundaries.

The main problem of Decision Trees is their **high variance**.
How can we fix this?

Decision Trees: Pros and Cons

Advantages

- **Highly interpretable**.
- Can handle numerical and categorical data.
- No feature scaling (e.g., normalization) is required.
- Fast for prediction (just follow a path).

Disadvantages

- **High Variance:** Very prone to overfitting. A small change in data can lead to a completely different tree.
- Greedy algorithm finds a local optimum, not guaranteed to be the *best* tree.
- Can struggle with non-axis-aligned boundaries.

The main problem of Decision Trees is their **high variance**.
How can we fix this? Ensemble Learning!

More trees help. Less correlation helps too.

$$\mathrm{Var}\left(\frac{1}{T}\sum_{t=1}^{T} h_t(x)\right) = \underbrace{\rho\sigma^2}_{\text{shared variation}} + \underbrace{\frac{(1-\rho)\sigma^2}{T}}_{\text{shrinks with more trees}}.$$

More trees help. Less correlation helps too.

$$\operatorname{Var}\left(\frac{1}{T}\sum_{t=1}^{T} h_t(x)\right) = \underbrace{\rho\sigma^2}_{\text{shared variation}} + \underbrace{\frac{(1-\rho)\sigma^2}{T}}_{\text{shrinks with more trees}}.$$

Bootstrap samples + random candidate features

Bagging by hand: five samples, three trees

Training samples: $A \quad B \quad C \quad D \quad E$ $\quad N = 5$; draw five times with replacement.

Bootstrap sample		Fit separately		Same query x_*
$A \quad A \quad C \quad D \quad E$	$\longrightarrow$	Tree 1	$\longrightarrow$	$\hat{y}_1 = 8$

Bagging by hand: five samples, three trees

Training samples: $A \quad B \quad C \quad D \quad E$ $N = 5$; draw five times with replacement.

Bootstrap sample	Fit separately	Same query x_*
$A \quad A \quad C \quad D \quad E$	Tree 1	$\hat{y}_1 = 8$
$B \quad B \quad C \quad E \quad E$	Tree 2	$\hat{y}_2 = 11$
$A \quad B \quad C \quad C \quad D$	Tree 3	$\hat{y}_3 = 11$

Bagging by hand: five samples, three trees

Training samples: $A \quad B \quad C \quad D \quad E$ $\quad N = 5$; draw five times with replacement.

Bootstrap sample	Fit separately	Same query x_*
$A \quad A \quad C \quad D \quad E$	Tree 1	$\hat{y}_1 = 8$
$B \quad B \quad C \quad E \quad E$	Tree 2	$\hat{y}_2 = 11$
$A \quad B \quad C \quad C \quad D$	Tree 3	$\hat{y}_3 = 11$

Regression: $\hat{y}(x_*) = (8 + 11 + 11)/3 = 10$.

Random forests add randomness at each split

Bagging + random candidate features

1. Draw a bootstrap sample.

Random forests add randomness at each split

Bagging + random candidate features

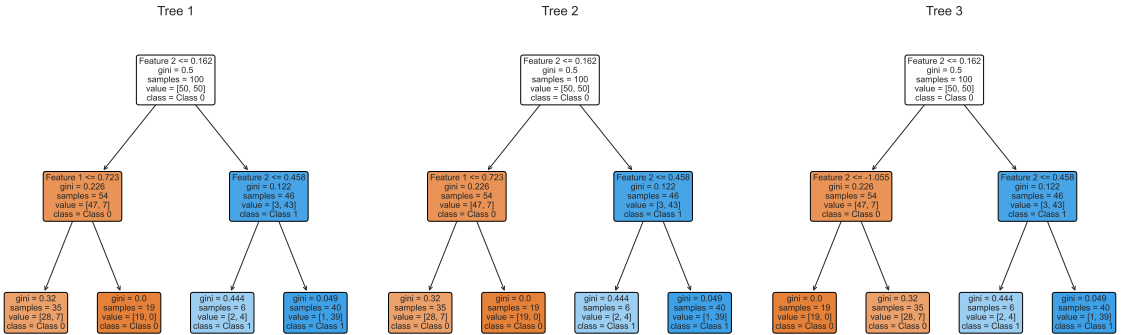
1. Draw a bootstrap sample.
2. At each node, sample m features; choose the best split among them.

Random forests add randomness at each split

Bagging + random candidate features

1. Draw a bootstrap sample.
2. At each node, sample m features; choose the best split among them.
3. Repeat for T trees, then aggregate.

Random Forests: Example



Each tree leaves some observations out

$$P(\text{not sampled}) = \left(1 - \frac{1}{N}\right)^N \approx e^{-1} \approx 37\%.$$

1. For each observation, collect trees that did not train on it.
2. Predict with those trees only.
3. Compare the OOB predictions with the observed labels.

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. **Mean Decrease in Impurity (MDI)**
2. **Permutation Importance (MDA)**

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. Mean Decrease in Impurity (MDI)

- When training, measure the total impurity reduction (Gini/MSE) from all splits on a feature f_j.

2. Permutation Importance (MDA)

- A more robust method.

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. Mean Decrease in Impurity (MDI)

- When training, measure the total impurity reduction (Gini/MSE) from all splits on a feature f_j.
- Average this reduction over all trees in the forest.

2. Permutation Importance (MDA)

- A more robust method.
  1. Calculate the OOB error for the forest.

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. Mean Decrease in Impurity (MDI)

- When training, measure the total impurity reduction (Gini/MSE) from all splits on a feature f_j.
- Average this reduction over all trees in the forest.
- **Pros:** Fast (calculated during training).

2. Permutation Importance (MDA)

- A more robust method.
  1. Calculate the OOB error for the forest.
  2. For a feature f_j: **shuffle** (permute) its values in the OOB data.

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. Mean Decrease in Impurity (MDI)

- When training, measure the total impurity reduction (Gini/MSE) from all splits on a feature f_j.
- Average this reduction over all trees in the forest.
- **Pros:** Fast (calculated during training).
- **Cons:** Biased towards high-cardinality features.

2. Permutation Importance (MDA)

- A more robust method.
  1. Calculate the OOB error for the forest.
  2. For a feature f_j: **shuffle** (permute) its values in the OOB data.
  3. Recalculate OOB error with the shuffled data.

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. Mean Decrease in Impurity (MDI)

- When training, measure the total impurity reduction (Gini/MSE) from all splits on a feature f_j.
- Average this reduction over all trees in the forest.
- **Pros:** Fast (calculated during training).
- **Cons:** Biased towards high-cardinality features.

2. Permutation Importance (MDA)

- A more robust method.
  1. Calculate the OOB error for the forest.
  2. For a feature f_j: **shuffle** (permute) its values in the OOB data.
  3. Recalculate OOB error with the shuffled data.
- The *drop* in performance is the “importance” of f_j.

Random Forests: Feature Importance

Since RF is a “black box” how do we know which features are important?

1. Mean Decrease in Impurity (MDI)

- When training, measure the total impurity reduction (Gini/MSE) from all splits on a feature f_j.
- Average this reduction over all trees in the forest.
- **Pros:** Fast (calculated during training).
- **Cons:** Biased towards high-cardinality features.

2. Permutation Importance (MDA)

- A more robust method.
  1. Calculate the OOB error for the forest.
  2. For a feature f_j: **shuffle** (permute) its values in the OOB data.
  3. Recalculate OOB error with the shuffled data.
- The *drop* in performance is the “importance” of f_j.
- **Pros:** More reliable, less biased.

What must we choose for each predictor?

Method	Key choice	Watch for
KNN	Neighbors and distance	Scale, irrelevant features
Tree	Depth and leaf size	Unstable partitions
Forest	Feature sampling and trees	Correlated predictions

Different regions. The same generalization question.

Neighbors	Regions	Ensembles
Choose K	Control tree size	Reduce correlation

Does it predict well on unseen data?

Reading materials & Reference

Reading materials:

- Chapter 16, 18, Probabilistic Machine Learning: an introduction.
- Chapter 8, The Elements of Statistical Learning (Hastie, Tibshirani, Friedman).
- EPFL, CS-433 Machine Learning, Lecture on k-NN.

Optional: derive the local-coverage formula

For independent uniform inputs in $[0,1]^d$, choose a cube fully contained in the domain, with side r and volume r^d.

$$P(\text{one point outside}) = 1 - r^d,$$

Optional: derive the local-coverage formula

For independent uniform inputs in $[0,1]^d$, choose a cube fully contained in the domain, with side r and volume r^d.

$$P(\text{one point outside}) = 1 - r^d,$$
$$P(\text{all } N \text{ outside}) = (1 - r^d)^N,$$

Optional: derive the local-coverage formula

For independent uniform inputs in $[0,1]^d$, choose a cube fully contained in the domain, with side r and volume r^d.

$$
\begin{aligned}
P(\text{one point outside}) &= 1 - r^d, \\
P(\text{all } N \text{ outside}) &= (1 - r^d)^N, \\
P(\text{at least one inside}) &= 1 - (1 - r^d)^N.
\end{aligned}
$$

Optional: derive the local-coverage formula

For independent uniform inputs in $[0,1]^d$, choose a cube fully contained in the domain, with side r and volume r^d.

$$
\begin{aligned}
P(\text{one point outside}) &= 1 - r^d, \\
P(\text{all } N \text{ outside}) &= (1 - r^d)^N, \\
P(\text{at least one inside}) &= 1 - (1 - r^d)^N.
\end{aligned}
$$

Set the last expression to $1/2$:

$$
(1 - r^d)^N = \frac{1}{2} \quad \Longrightarrow \quad r = \left(1 - 2^{-1/N}\right)^{1/d}.
$$

Deeper Dive: Generalization bound for 1-NN (Setup)

Let's formalize the problem for binary classification $\mathcal{Y} = \{0, 1\}$.

- **Setup:** $(X, Y) \sim \mathcal{D}$ over $\mathcal{X} \times \mathcal{Y} = [0, 1]^d \times \{0, 1\}$

Deeper Dive: Generalization bound for 1-NN (Setup)

Let's formalize the problem for binary classification $\mathcal{Y} = \{0, 1\}$.

- **Setup:** $(X, Y) \sim \mathcal{D}$ over $\mathcal{X} \times \mathcal{Y} = [0, 1]^d \times \{0, 1\}$
- **Goal:** Bound the classification error:

$$L(f) = \mathbb{P}_{(X,Y)\sim\mathcal{D}}(Y \neq f(X))$$

Deeper Dive: Generalization bound for 1-NN (Setup)

Let's formalize the problem for binary classification $\mathcal{Y} = \{0, 1\}$.

- **Setup:** $(X, Y) \sim \mathcal{D}$ over $\mathcal{X} \times \mathcal{Y} = [0, 1]^d \times \{0, 1\}$
- **Goal:** Bound the classification error:

$$L(f) = \mathbb{P}_{(X,Y)\sim\mathcal{D}}(Y \neq f(X))$$

- Let $\eta(x) = \mathbb{P}\{Y = 1 | X = x\}$ be the true, unknown conditional probability of class 1.

Deeper Dive: Generalization bound for 1-NN (Setup)

Let's formalize the problem for binary classification $\mathcal{Y} = \{0, 1\}$.

- **Setup:** $(X, Y) \sim \mathcal{D}$ over $\mathcal{X} \times \mathcal{Y} = [0, 1]^d \times \{0, 1\}$
- **Goal:** Bound the classification error:

$$L(f) = \mathbb{P}_{(X,Y)\sim\mathcal{D}}(Y \neq f(X))$$

- Let $\eta(x) = \mathbb{P}\{Y = 1 | X = x\}$ be the true, unknown conditional probability of class 1.
- **Baselines:**
  - The Bayes Classifier, $f_*(x)$, is the *optimal* possible classifier. It predicts the most likely class:

$$f_*(x) = \mathbf{1}_{\{\eta(x) > 1/2\}}$$

Deeper Dive: Generalization bound for 1-NN (Setup)

Let's formalize the problem for binary classification $\mathcal{Y} = \{0, 1\}$.

- **Setup:** $(X, Y) \sim \mathcal{D}$ over $\mathcal{X} \times \mathcal{Y} = [0, 1]^d \times \{0, 1\}$
- **Goal:** Bound the classification error:

$$L(f) = \mathbb{P}_{(X,Y)\sim\mathcal{D}}(Y \neq f(X))$$

- Let $\eta(x) = \mathbb{P}\{Y = 1 | X = x\}$ be the true, unknown conditional probability of class 1.
- **Baselines:**
  - The Bayes Classifier, $f_*(x)$, is the *optimal* possible classifier. It predicts the most likely class:

$$f_*(x) = \mathbf{1}_{\{\eta(x)>1/2\}}$$

  - The error of this optimal classifier is the Bayes Risk, $\mathcal{L}(f_*)$:

$$\mathcal{L}(f_*) = \mathbb{E}_x[\min\{\eta(x), 1 - \eta(x)\}]$$

    This is the absolute minimum, irreducible error for the problem.

Proof of Bayes Classifier

Let $\eta(x) = \mathbb{P}\{Y = 1|X = x\}$ be the true, unknown conditional probability of class 1.

$$\begin{aligned}\eta(x) \geq 1/2 &\iff \mathbb{P}(Y = 1|X = x) \geq 1/2 \\ &\iff \mathbb{P}(Y = 1|X = x) \geq \mathbb{P}(Y = 0|X = x) \\ &\iff 1 \in \underset{y\in\{0,1\}}{\operatorname{argmax}}\, \mathbb{P}(Y = y|X = x)\end{aligned}$$

Thus $\mathbf{1}_{\eta(x)\geq 1/2} = \operatorname{argmax}_{y\in\{0,1\}} \mathbb{P}(Y = y|X = x) = f_*(x)$

Proof of Bayes Risk Formula

Let $\eta(x) = \mathbb{P}\{Y = 1 | X = x\}$ be the true, unknown conditional probability of class 1.

$$
\begin{aligned}
\mathcal{L}(f_*) &= \mathbb{E}_{(X,Y)\sim\mathcal{D}}[\mathbf{1}_{f_*(X)\neq Y}] \\
&= \mathbb{E}_{X\sim\mathcal{D}_X}[\mathbb{E}_{Y\sim\mathcal{D}_{Y|X}}[\mathbf{1}_{f_*(X)\neq Y}|X]] \\
&= \mathbb{E}_{X\sim\mathcal{D}_X}[\mathbb{E}_{Y\sim\mathcal{D}_{Y|X}}[\mathbf{1}_{f_*(X)\neq Y}|X]\mathbf{1}_{\eta(X)\geq 1/2} + \mathbb{E}_{Y\sim\mathcal{D}_{Y|X}}[\mathbf{1}_{f_*(X)\neq Y}|X]\mathbf{1}_{\eta(X)<1/2}] \\
&= \mathbb{E}_{X\sim\mathcal{D}_X}[\mathbb{E}_{Y\sim\mathcal{D}_{Y|X}}[\mathbf{1}_{1\neq Y}|X]\mathbf{1}_{\eta(X)\geq 1/2} + \mathbb{E}_{Y\sim\mathcal{D}_{Y|X}}[\mathbf{1}_{0\neq Y}|X]\mathbf{1}_{\eta(X)<1/2}] \\
&= \mathbb{E}_{X\sim\mathcal{D}_X}[\mathbb{P}(Y = 0|X)\mathbf{1}_{\eta(X)\geq 1/2} + \mathbb{P}(Y = 1|X)\mathbf{1}_{\eta(X)<1/2}] \\
&= \mathbb{E}_{X\sim\mathcal{D}_X}[\min\{\eta(X), 1 - \eta(X)\}]
\end{aligned}
$$

Deeper Dive: Generalization bound for 1-NN (The Bound)

- **Key Assumption:** We need to assume that the problem is "learnable". We assume the probability function $\eta(x)$ is Lipschitz continuous:

$$|\eta(x) - \eta(x')| \leq c||x - x'||$$

(In words: if two points x and x' are close, their probabilities $\eta(x)$ and $\eta(x')$ must also be close.)

Deeper Dive: Generalization bound for 1-NN (The Bound)

- **Key Assumption:** We need to assume that the problem is "learnable". We assume the probability function $\eta(x)$ is Lipschitz continuous:

$$|\eta(x) - \eta(x')| \leq c||x - x'||$$

(In words: if two points x and x' are close, their probabilities $\eta(x)$ and $\eta(x')$ must also be close.)

- **The 1-NN Bound:** Let $\mathcal{L}(f_{S_{train}})$ be the risk of our 1-NN classifier trained on dataset S_{train}. It can be shown that:

$$\mathbb{E}_{S_{train}}[\mathcal{L}(f_{S_{train}})] \leq 2\mathcal{L}(f_*) + c \cdot \mathbb{E}_{S_{train},x}[||x - \text{nbh}_{S_{train},1}(x)||]$$

Deeper Dive: Generalization bound for 1-NN (The Bound)

- **Key Assumption:** We need to assume that the problem is "learnable". We assume the probability function $\eta(x)$ is Lipschitz continuous:

$$|\eta(x) - \eta(x')| \leq c||x - x'||$$

(In words: if two points x and x' are close, their probabilities $\eta(x)$ and $\eta(x')$ must also be close.)

- **The 1-NN Bound:** Let $\mathcal{L}(f_{S_{train}})$ be the risk of our 1-NN classifier trained on dataset S_{train}. It can be shown that:

$$\mathbb{E}_{S_{train}}[\mathcal{L}(f_{S_{train}})] \leq 2\mathcal{L}(f_*) + c \cdot \mathbb{E}_{S_{train},x}[||x - \text{nbh}_{S_{train},1}(x)||]$$

- The 1-NN error is bounded by:
  - (Twice the optimal Bayes risk) + (A "geometric term" that depends on the average distance to the nearest neighbor).

Deeper Dive: 1-NN Analysis (Interpretation)

What does the bound $\mathbb{E}[\mathcal{L}] \leq 2\mathcal{L}(f_*) +$ (geometric term) tell us?

- **Limit 1: As $N \to \infty$ (fixed d)**
  - The average distance to the nearest neighbor $\to 0$.
  - The geometric term vanishes.

$$\mathbb{E}_{S_{train}}[\mathcal{L}(f_{S_{train}})] \leq 2\mathcal{L}(f_*)$$

  - **Takeaway:** With infinite data, the 1-NN error is at most *twice the optimal (Bayes) error*. This upper bound alone does not establish Bayes consistency or inconsistency.

Deeper Dive: 1-NN Analysis (Interpretation)

What does the bound $\mathbb{E}[\mathcal{L}] \leq 2\mathcal{L}(f_*) +$ (geometric term) tell us?

- **Limit 1: As $N \to \infty$ (fixed d)**
  - The average distance to the nearest neighbor $\to 0$.
  - The geometric term vanishes.

$$\mathbb{E}_{S_{train}}[\mathcal{L}(f_{S_{train}})] \leq 2\mathcal{L}(f_*)$$

  - **Takeaway:** With infinite data, the 1-NN error is at most *twice the optimal (Bayes) error*. This upper bound alone does not establish Bayes consistency or inconsistency.

- **Limit 2: As $d \to \infty$ (fixed N)**
  - A covering argument gives an upper bound of order $c\sqrt{d}N^{-1/(d+1)}$ for the geometric term.
  - To keep this sufficient upper bound small, its sample requirement grows rapidly with d.
  - **Takeaway:** The guarantee weakens with dimension. A growing upper bound does not prove that the actual classification error grows.

Optional: other split criteria

How do we measure Impurity$(\mathcal{D})$ for a node with class proportions p_k?

For Classification

For Regression

Optional: other split criteria

How do we measure Impurity$(\mathcal{D})$ for a node with class proportions p_k?

For Classification

- Gini Impurity
  - Probability that two independent class draws disagree.
  - $G(p) = 1 - \sum_{k=1}^{K} p_k^2$
  - Max impurity = 0.5 (for 2 classes). Pure = 0.

For Regression

Optional: other split criteria

How do we measure Impurity$(\mathcal{D})$ for a node with class proportions p_k?

For Classification

- Gini Impurity
  - Probability that two independent class draws disagree.
  - $G(p) = 1 - \sum_{k=1}^{K} p_k^2$
  - Max impurity = 0.5 (for 2 classes). Pure = 0.
- Entropy
  - Measures uncertainty (from information theory).
  - $H(p) = -\sum_{k=1}^{K} p_k \log_2 p_k$
  - Max impurity = 1.0 (for 2 classes). Pure = 0.

For Regression

Optional: other split criteria

How do we measure Impurity$(\mathcal{D})$ for a node with class proportions p_k?

For Classification

- Gini Impurity
  - Probability that two independent class draws disagree.
  - $G(p) = 1 - \sum_{k=1}^{K} p_k^2$
  - Max impurity = 0.5 (for 2 classes). Pure = 0.
- Entropy
  - Measures uncertainty (from information theory).
  - $H(p) = -\sum_{k=1}^{K} p_k \log_2 p_k$
  - Max impurity = 1.0 (for 2 classes). Pure = 0.

For Regression

- Mean Squared Error (MSE)
  - Measures variance of the target variable y.
  - MSE $= \frac{1}{N} \sum_{i=1}^{N} (y_i - \bar{y})^2$
  - Goal is to minimize variance in the leaves.